Neuro-Symbolic Causal Inference

via Language-Model Knowledge Extraction and Bayesian Networks

Project 1 — Phase 1: Machine Learning Evaluation

CSAI Student Project — Applied Machine Learning & Deep Learning

Dataset	Corr2Cause (Jin et al., ICLR 2024), corrected labels
Framework	scikit-learn, NetworkX, pgmpy
Institution	Newton School of Technology, Rishihood University
Repository	https://github.com/HemanthTenneti/AML-DL-Project-2026

Team Members

Krishnadevan Manju Liby
Hemanth Tenneti
Arina Ali
Manan Bansal
Aaloke Eppalapalli

October 2026

Contents

1 Abstract 2

2 Introduction 2

3 Background and Literature Review 3
- 3.1 Causal reasoning with graphs . . . 3
- 3.2 Causal discovery from independence facts . . . 3
- 3.3 Causal relations in text and the Corr2Cause benchmark . . . 4
- 3.4 Positioning . . . 4

4 Dataset and Exploratory Analysis 4
- 4.1 Corr2Cause in one example . . . 4
- 4.2 Splits, robustness sets and leakage . . . 5
- 4.3 Label quality . . . 5
- 4.4 What the data looks like . . . 5

5 Theoretical Foundations 6

6 Methodology 7
- 6.1 Stage 1: Text to facts . . . 7
- 6.2 Stage 2: Facts to a causal graph . . . 7
- 6.3 Stage 3: Feature engineering . . . 8
- 6.4 Stage 4: Classifiers . . . 8
- 6.5 Stage 5: From triples to answers . . . 9

7 Experiments and Results 9
- 7.1 Ablation: reasoning, not wording, carries the signal . . . 9
- 7.2 The final model is a readable rule . . . 10
- 7.3 Comparison and robustness . . . 10

8 Graph Stage and Causal Queries 12
- 8.1 Edge recovery . . . 12
- 8.2 Intervention queries . . . 12
- 8.3 Why the Bayesian network matters: seeing vs doing . . . 13

9 Error Analysis and Limitations 13

10 Reproducibility and Code Quality 14

11 Road to Phase 2 15

12 Conclusion 16

13 Team Contributions 16

1. Abstract

Language models are good at sounding causal and bad at *being* causal: given a list of correlations, they often claim causes that the data cannot support. This project separates the two jobs. A *language layer* turns text into structured statistical facts; a *reasoning layer* applies textbook causal-discovery rules (the PC algorithm) and an interpretable classifier to decide which causal claims are actually entailed; a *graph layer* assembles the accepted cause→effect triples into an acyclic graph, fits a Bayesian network with `pgmpy`, and answers intervention queries.

In Phase 1 every layer is classical machine learning. On the Corr2Cause benchmark our Rules + PC + Decision-Tree pipeline reaches an F1 of **0.997** on the held-out test set (text-only TF-IDF baseline: 0.655), stays at **0.997** when all variable names are changed (baseline: 0.322), recovers **73/73** gold cause edges with no false edges, and answers **97.5%** of 242 known intervention queries correctly. An ablation shows that the reasoning features, not the wording, carry the signal. The one clear failure, *paraphrased* hypotheses that the hand-written rules cannot read (F1 = 0), is the gap that Phase 2 targets with DistilBERT.

2. Introduction

News articles and scientific abstracts are full of statements such as "X is associated with Y". Turning such statements into reliable *causal* conclusions is hard, and it is exactly where large language models stumble: on the Corr2Cause benchmark, which asks whether a causal claim follows from a set of correlations, the original study found that even strong fine-tuned transformers mostly pattern-match and fall apart when the wording or variable names change [3].

The project brief asks for a system that *extracts candidate cause–effect relations from text and then represents and tests them in a Bayesian network*, keeping "fluent language understanding" separate from "formal causal reasoning and counterfactual queries". We take that separation literally and build a three-layer pipeline with fixed interfaces, so that Phase 2 can replace a single layer and be compared fairly with Phase 1.

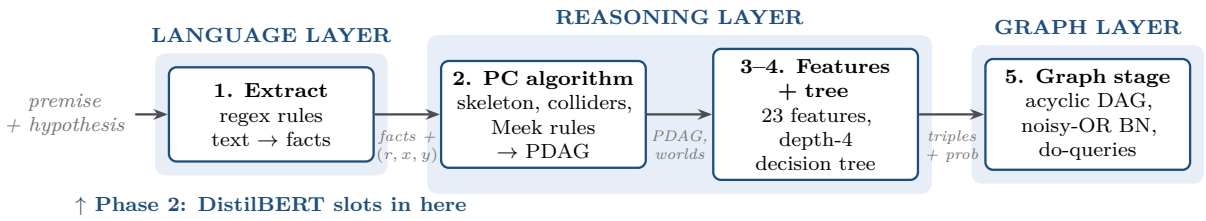


Figure 1: The Phase 1 pipeline. Each layer has one job and a fixed interface. The language layer is the only part Phase 2 will replace, so the reasoning and graph layers (and the evaluation code) stay identical.

Contributions of Phase 1.

- A transparent, end-to-end ML pipeline that goes from raw text to intervention answers (Fig. 1).
- An implementation of the PC orientation rules plus enumeration of *equivalent DAGs*, giving a "support" feature that measures how many causal worlds agree with a claim.
- A feature ablation, a text-only baseline, and two robustness tests (renamed variables, reworded hypotheses).
- A graph stage that enforces acyclicity, quantifies effects with a Bayesian network, and shows numerically why correlation and intervention differ.
- A reproducible repository with unit tests and an interactive pipeline tracer (Section 10).

Table 1: Where each Phase 1 evaluation criterion is addressed.

Criterion	Section(s)
Literature review	Section 3
Dataset quality & EDA	Section 4 (splits, leakage check, label correction, class balance)
Feature engineering	Section 6.3 (23 features in 4 groups) and the ablation in Section 7.1
Theoretical rigour	Section 5 (Markov equivalence, support criterion, do-calculus)
Model application	Sections 7 and 8
GitHub repository & code quality	Section 10

3. Background and Literature Review

3.1. Causal reasoning with graphs

Pearl's work on Bayesian networks [1] gave a compact way to encode a joint distribution as a directed acyclic graph (DAG) plus local conditional probability tables, with *d-separation* linking graph structure to conditional independence. *Causality* [7] added the do(·) operator: $P(Y \mid \mathrm{do}(X{=}x))$ is what happens when we *set* X, which can differ sharply from $P(Y \mid X{=}x)$, what we see when we merely *observe* it.

3.2. Causal discovery from independence facts

If all conditional independences of a system are known, the PC algorithm [4] recovers the graph's *skeleton* and the direction of every edge that the data can identify. Verma and Pearl [6] showed that two DAGs are indistinguishable from observational data exactly when they share a skeleton and their v-structures (colliders $X \to Z \leftarrow Y$); Meek [5] gave the complete set of rules for orienting the remaining edges. The output is a partially directed graph (PDAG) that stands for a whole *Markov equivalence class* of DAGs.

3.3. Causal relations in text and the Corr2Cause benchmark

Classical causal relation extraction relies on lexical and dependency patterns ("X leads to Y", "due to") [9, 10]; it is precise on the templates it knows and blind to everything else. Transformers such as BERT [2] and its distilled version DistilBERT [8] learn wording from data and are now standard for relation classification. Corr2Cause [3] tests whether such models can *reason*: each premise lists every correlation and independence in a small closed system, and the model must judge whether a causal hypothesis is entailed. The authors report that off-the-shelf LLMs score near chance and that fine-tuned models fail under paraphrasing and variable renaming, a classic case of shortcut learning [11, 12]. Surveys of LLM causal reasoning reach a similar conclusion: fluent causal language is not the same as causal inference [13].

3.4. Positioning

Neuro-symbolic AI argues for pairing learned perception with explicit reasoning [14]. Our design follows this line (Table 2): the language layer handles wording, while acyclicity, uncertainty and interventions are guaranteed by symbolic machinery rather than hoped for from a text model.

Table 2: How the main approaches compare, and what we take from each.

Approach	**Strength**	**Limitation / our use**
Pattern-based extraction [9, 10]	Precise, explainable	Brittle to rewording. *Our Phase 1 language layer.*
PC algorithm + Meek rules [4, 5]	Provably sound orientation	Needs independence facts as input. *Our reasoning core.*
Bayesian networks, do-calculus [1, 7]	Uncertainty and interventions	Needs a DAG. *Our graph layer (pgmpy [15]).*
End-to-end transformers [2, 8]	Robust to wording	Shortcut-prone at reasoning [3]. *Phase 2 language layer.*

4. Dataset and Exploratory Analysis

4.1. Corr2Cause in one example

Each row pairs a *premise* (all statistical relations in a closed system of 2–6 variables) with one *hypothesis* and a binary label: 1 if the claim *must* hold, 0 otherwise. A real test example:

> **Premise.** Suppose there is a closed system of 3 variables, A, B and C. All the statistical relations among these 3 variables are as follows: A correlates with C. B correlates with C. However, A is independent of B.
>
> **Hypotheses.** "A directly causes C." (label 1); "B directly causes A." (label 0); "There exists at least one collider of B and C." (label 0).

The reasoning: A and C are dependent, B and C are dependent, but A and B are independent. The only structure consistent with this is the collider $A \to C \leftarrow B$, so "A directly causes C" is entailed.

We chose Corr2Cause, the starter dataset the brief recommends, because every label is *provably* correct with respect to the premise. That lets us measure extraction and reasoning exactly, which is not possible with open news text, where the "true" causal graph is unknown.

4.2. Splits, robustness sets and leakage

Table 3 summarises the data. Many hypotheses share a premise, so a random row split would leak near-duplicate text between train and test. Following the brief ("split documents before extraction"), we use the official **premise-grouped** split and verified that train and test share **0 premises**. Dev shares 2 of its 22 premises with train, so we treat dev as a secondary check and report headline numbers on test.

Two official robustness sets reuse the 21 test premises: *refactorization* renames variables (e.g. A, B and C become Z, Y and X) and *paraphrase* rewords the hypotheses ("directly causes" → "directly affects").

Table 3: Data used in Phase 1 (corrected labels).

Split	Rows	Premises	% valid	Purpose
train	205,734	2,338	21.0	fit all models
dev	1,076	22	14.8	secondary check
test	1,162	21	16.5	headline results
refactorization	2,246	21	16.2	robustness: renamed variables
paraphrase	2,246	21	16.2	robustness: reworded hypotheses

4.3. Label quality

The original Corr2Cause generator contained a bug that wrote 6,943 wrong labels (3.3%). We use the publicly released corrected labels [16], which keep the text identical. The fix changes 6,897 train, 28 dev and 18 test labels. We noticed the problem ourselves when our PC reasoner kept "disagreeing" with the original labels on whole premises; for transparency, Section 7 also reports test scores on the original labels.

4.4. What the data looks like

Three EDA findings shaped the design:

- **Imbalance.** A model that always says "not valid" gets about 84% accuracy on test, which is meaningless. F1 of the valid class is the primary metric throughout.
- **Size shift.** Train is dominated by 6-variable graphs, test is not. Features that depend on graph *structure* rather than raw counts should transfer better.
- **Names carry order.** Variables are lettered in causal order in the generator, so "a later letter causes an earlier one" is nearly always false. A text model can exploit this shortcut, and the refactorization set exists to catch it.

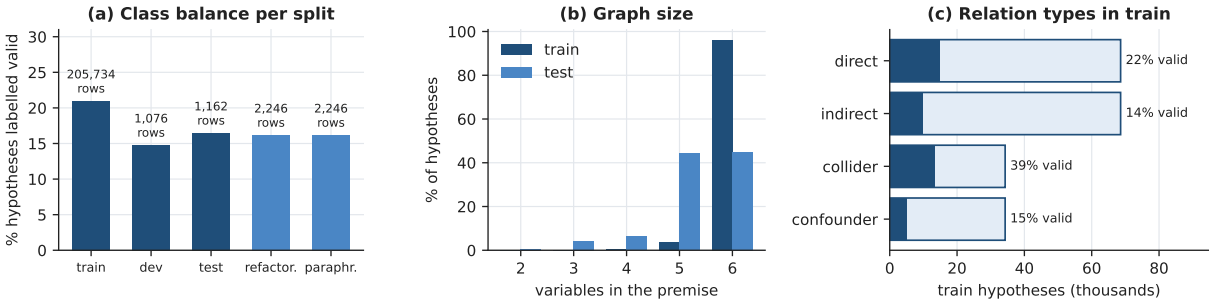


Figure 2: (a) Only 15–21% of hypotheses are valid in every split, so we report F1 of the valid class rather than accuracy. (b) Training premises are almost all 6-variable systems, while test spans 2–6 variables: a mild distribution shift that rewards methods that generalise across graph sizes. (c) Each premise is asked about every relation type for every variable pair; collider claims are valid far more often than the others.

5. Theoretical Foundations

This section collects the theory the pipeline relies on. Each result is short, but together they explain why the system is correct by construction rather than by luck.

T1: Faithfulness and the skeleton

In a faithful causal DAG, two variables X, Y are *adjacent* if and only if no set S of other variables makes them conditionally independent ($X \not\perp\!\!\!\perp Y \mid S$ for all S). Corr2Cause lists every independence, so the skeleton is read off directly: join X and Y when they have no separating set.

T2: Colliders are identifiable; most other edges are not

For a path $X - Z - Y$ with X, Y non-adjacent, Z is a collider ($X \to Z \leftarrow Y$) iff Z belongs to *no* separating set of (X, Y). Two DAGs are observationally equivalent iff they share the skeleton and the v-structures [6]. Meek's rules R1–R3 [5] then orient every edge whose reversal would create a cycle or a new collider. Edges still undirected after that are genuinely unidentifiable.

T3: The validity criterion ("support")

Let $\mathcal{E}(G)$ be the set of DAGs represented by the PDAG G. For a hypothesis h (e.g. "X directly causes Y"),

$$\text{supp}(h) \;=\; \frac{1}{|\mathcal{E}(G)|} \sum_{D \in \mathcal{E}(G)} \mathbf{1}[\, h \text{ holds in } D \,].$$

Corr2Cause labels a claim valid iff it is implied by the premise, i.e. true in *every* equivalent DAG, so h is valid $\Leftrightarrow \text{supp}(h) = 1$. A claim with support 0.67 is *possible* but not *entailed*.

T4: Interventions in a Bayesian network

A DAG with conditional tables factorises as $P(v_1, \ldots, v_n) = \prod_i P(v_i \mid \mathrm{pa}_i)$. Setting $X{=}x$ deletes X's own factor (*truncated factorisation* [7]): $P(\mathbf{v} \mid \mathrm{do}(X{=}x)) = \prod_{i:V_i \neq X} P(v_i \mid \mathrm{pa}_i)\big|_{X=x}$. We report the average causal effect ACE $= P(Y{=}1 \mid \mathrm{do}(X{=}1)) - P(Y{=}1 \mid \mathrm{do}(X{=}0))$. When X and Y share a cause Z, the observational difference $P(Y \mid X{=}1) - P(Y \mid X{=}0)$ also contains the $X \leftarrow Z \rightarrow Y$ path and therefore overstates the effect (Section 8.3).

6. Methodology

6.1. Stage 1: Text to facts

We first fixed a small **entity schema**. Variables are single capital letters, and premise facts take two forms, `correlates(X,Y)` and `independent(X,Y|S)`. Hypotheses are one of four relations (Table 4). One regular expression per sentence template fills this schema. Every fact keeps its premise id, and self-loops (X causes X) are rejected.

Table 4: The four causal relation types (our entity schema).

Relation	Template wording	Meaning in a DAG
direct	"X directly causes Y."	$X \rightarrow Y$
indirect	"X causes something else which causes Y."	$X \rightarrow Z \rightarrow \cdots \rightarrow Y$
collider	"There exists at least one collider of X and Y."	$X \rightarrow \cdots \rightarrow Z \leftarrow \cdots \leftarrow Y$
confounder	"There exists at least one confounder of X and Y."	$X \leftarrow \cdots \leftarrow Z \rightarrow \cdots \rightarrow Y$

```
HYPOTHESIS_PATTERNS = [  # src/causal_nlp/parsing.py (one rule per template)
  (re.compile(rf"^{VAR} directly causes {VAR}\.$"), DIRECT, 1, 2),
  (re.compile(rf"^{VAR} causes something else which causes {VAR}\.$"), INDIRECT, 1, 2),
  ...
]
def parse_hypothesis(text) -> Hypothesis | None:   # None = "unseen wording"
```

The parser returns a typed `Hypothesis(relation, x, y)` or `None`. This tiny interface is the contract Phase 2 must honour: a DistilBERT parser only needs to return the same tuple.

6.2. Stage 2: Facts to a causal graph

We implemented the three PC steps directly on the extracted facts (T1–T2): build the skeleton, orient v-structures, then apply Meek rules R1–R3 until nothing changes. To evaluate a hypothesis we then enumerate the equivalent DAGs: flip every undirected edge both ways and keep the combinations with no cycle and no extra collider. Figure 3 shows a real test premise with two undirected edges, giving $2^2 = 4$ candidate orientations, of which 3 are valid DAGs.

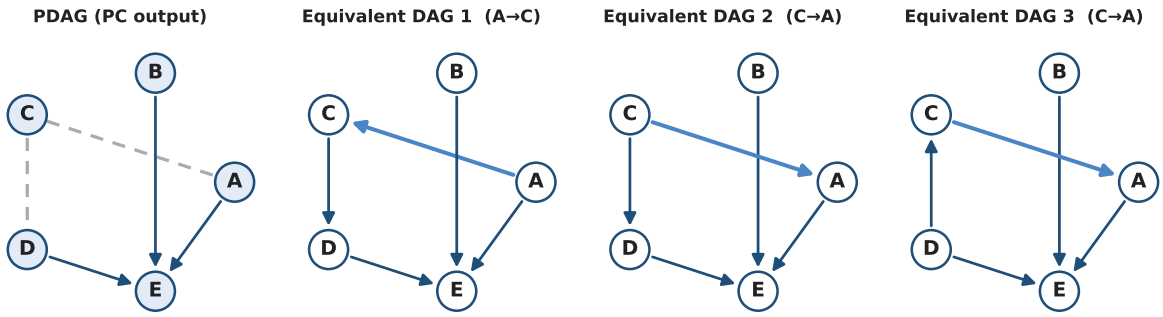


Figure 3: A 5-variable test premise. Left: the PDAG produced by our PC implementation (dashed = direction unknown). Right: its three equivalent DAGs. The edge between A and C points A→C in only one of them, so supp(A directly causes C) = 1/3 and the claim is *not* valid. By contrast, "C is an indirect cause of E" holds in all three worlds (support 1.0) and is valid.

6.3. Stage 3: Feature engineering

Each (premise, hypothesis) pair becomes a row of 23 numbers in four groups (Table 5). The groups mirror the reasoning steps, which lets the ablation measure how much each step adds. One design rule matters for robustness: **no feature uses a variable's name**, so renaming variables cannot change any input.

Table 5: Engineered features.

Group	#	Features (intuition)
Relation	4	one-hot of the claimed relation — *what is being claimed*
Text facts	8	no. of variables; X–Y correlated; adjacent in skeleton; no. of separating sets; size of smallest separating set; degree of X, Y; common neighbours — *read straight off the sentences*
PC graph	9	$X \to Y$, $Y \to X$, undirected; X ancestor of Y (definite / possible) and reverse; common children; common parents — *read off the PDAG*
Support	2	supp(h); number of equivalent DAGs — *how many possible worlds agree*

Building features for all 212,464 rows takes about 12 s on a laptop CPU, because each premise is parsed and analysed once and cached across its hypotheses.

6.4. Stage 4: Classifiers

- **Text-only baseline: TF-IDF + logistic regression.** Word 1–3-grams of premise and hypothesis (one-letter tokens kept so variable names are visible), class-balanced loss. No parsing and no reasoning. This is the fair ML counterpart of an end-to-end transformer.
- **Feature models:** logistic regression (standardised), a **depth-4 decision tree**, and a 300-tree random forest, each trained on the three cumulative feature groups.

We chose the decision tree as the final model. It matches the forest's accuracy, it can be printed and checked by hand, and it outputs a probability that the graph stage uses as an edge confidence. All models use seed 42.

6.5. Stage 5: From triples to answers

Only claims the classifier marks valid (`prob` $\geq$ `0.5`) reach the graph stage, which is how we implement the brief's "feed only reviewed predictions into the graph stage".

1. **Acyclic DAG.** Predicted *direct* edges are added from most to least confident; an edge that would close a cycle, or is a self-loop, is rejected and logged.
2. **Bayesian network.** Corr2Cause provides structure but no measurements, so each node receives illustrative *noisy-OR* tables: $P(V{=}1 \mid k \text{ active parents}) = 1-(1-0.1)(1-0.7)^k$ (base rate 10%, each cause fires with 70%). The network is built and checked in `pgmpy` [15].
3. **Intervention queries.** For each query "does do(X) change Y?" we compute the ACE (T4) with `CausalInference` and answer "yes" if ACE > 0.01.

Known queries are those whose answer is fixed by the gold labels or the text: *yes* if X directly or indirectly causes Y; *no* if Y causes X (an effect cannot cause its cause) or if X and Y are not even correlated.

7. Experiments and Results

Protocol. All models are trained on the full train split and evaluated on the untouched test set. The primary metric is **F1 of the valid class**; we also report precision and recall. Nothing is tuned on test.

7.1. Ablation: reasoning, not wording, carries the signal

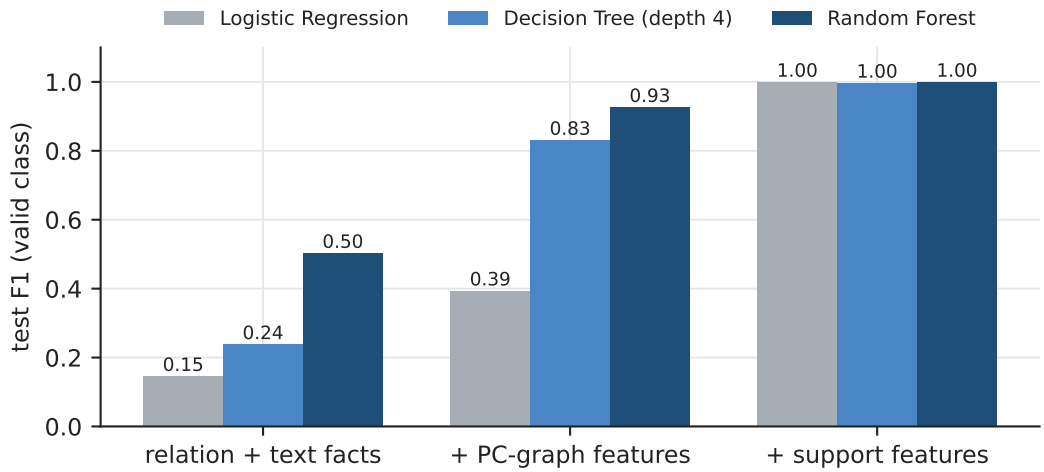


Figure 4: Test F1 as feature groups are added. Text facts alone are weak; PC-graph features give the largest jump; support makes the task essentially solved for every model.

Figure 4 tells the main story of Phase 1. With only facts read off the sentences, even a random forest reaches F1 of just 0.50, because validity depends on *combinations* of facts that a flat model cannot form. Adding the PC-graph features lifts the tree from 0.24 to 0.83 and the forest to 0.93. Adding support takes all three models to 0.997–1.000. Once the reasoning is done symbolically, the learning problem becomes almost linearly separable.

7.2. The final model is a readable rule

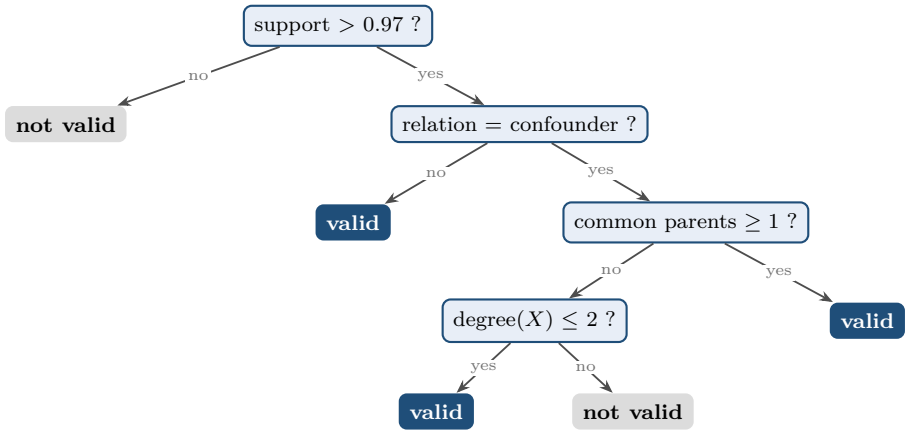


Figure 5: The learned depth-4 tree, simplified (the left subtree's three leaves all predict "not valid"). The tree *learned* the rule "valid iff support ≈ 1" from data, plus a small branch for confounders that absorbs a quirk in how the generator defines a common cause.

The tree in Fig. 5 rediscovers criterion T3 on its own, without being told. Support accounts for 99% of its Gini importance (Fig. 6a). The test confusion matrix (Fig. 6b) has exactly one error: a valid *confounder* claim the confounder branch rejects, giving confounder recall 0.93 (14 of 15). All other relations score F1 = 1.00.

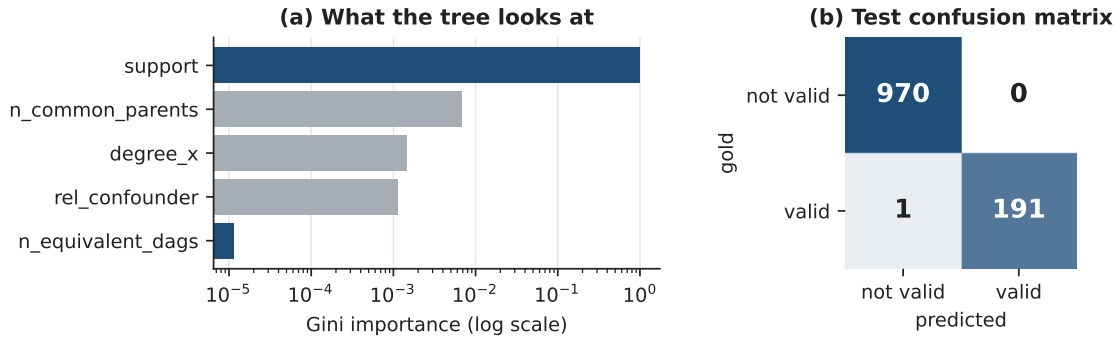


Figure 6: (a) Feature importance of the final tree (log scale). (b) Test confusion matrix: 970 true negatives, 191 true positives, 0 false positives, 1 false negative.

7.3. Comparison and robustness

Table 6 and Fig. 7 show three things:

- **In distribution**, the pipeline is near-perfect (0.997 vs 0.655). TF-IDF reaches high recall only by flagging many claims as valid, so its precision is about 0.5.
- **Under renaming**, TF-IDF collapses from 0.655 to 0.322. Its recall falls from 0.94 to 0.24, the signature of the letter-order shortcut noted in Section 4. Our features never see names, so the pipeline keeps 0.997.
- **On the original (buggy) labels** the pipeline still scores 0.949. The drop from 0.997 is consistent with the 18 test labels that the bug fix changed.

Table 6: Relation-extraction results, F1 of the valid class (precision / recall below each F1 on test).

System	dev	test	refactor.	paraphrase	test, orig. labels
TF-IDF + LR (text only)	0.606	0.655	0.322	0.250	0.625
P / R		0.50 / 0.94	0.49 / 0.24	0.27 / 0.23	
Rules + PC + Decision Tree	**1.000**	**0.997**	**0.997**	0.000	**0.949**
P / R		1.00 / 0.99	1.00 / 0.99	–	

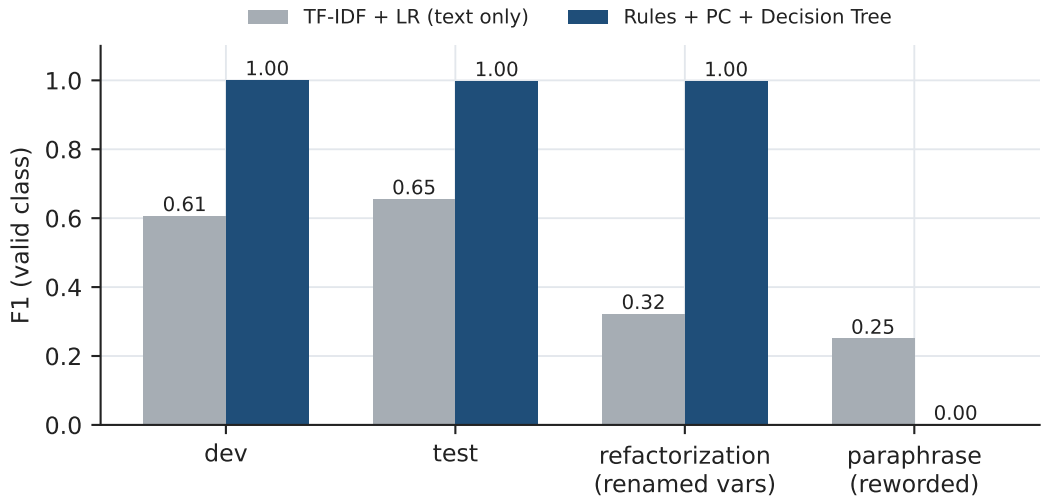


Figure 7: The text-only model halves its score when variables are renamed; the neuro-symbolic pipeline does not move. Both fail on reworded hypotheses, for different reasons (Section 9).

Key takeaway. Doing the causal reasoning symbolically turns a hard text-classification problem into an easy, interpretable one, and makes the result immune to the variable-name shortcut that defeats text-only models.

8. Graph Stage and Causal Queries

8.1. Edge recovery

From the classifier's direct-cause triples on the 21 test premises, the graph stage builds one DAG per premise. It predicts **73 edges, all 73 correct**, against 73 gold edges: edge precision = recall = **1.00**. Figure 8 shows the largest test premise.

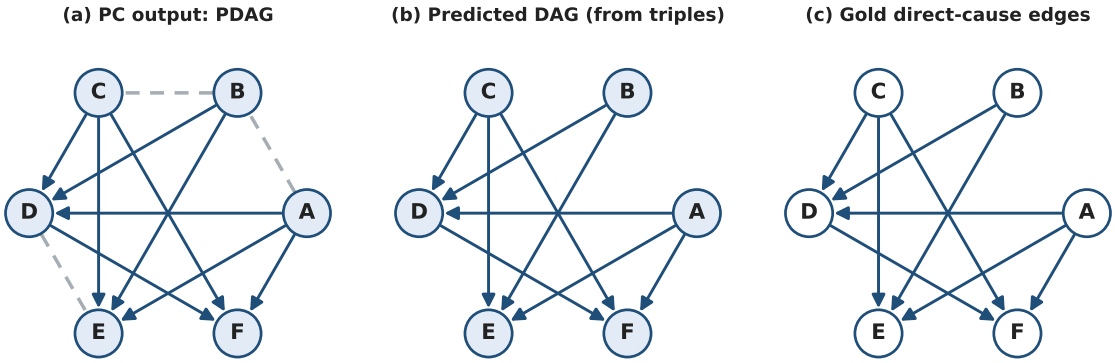


Figure 8: A 6-variable test premise. (a) The PC output leaves three edges undirected. (b) The DAG assembled from the classifier's accepted triples keeps only edges that are certain in every world. (c) It matches the gold direct-cause edges exactly.

Acyclicity check. Feeding the builder a contradictory candidate set, A→B (0.95), B→C (0.90), C→A (0.40), B→B (0.30), keeps the first two and rejects the cycle-closing C→A and the self-loop. Text extraction alone cannot guarantee this; the graph layer does, and it rejects the least confident edge first.

8.2. Intervention queries

The test premises yield **242 known queries** (spread over 14 of the 21 premises), and the Bayesian networks answer **97.5%** correctly (Table 7).

Table 7: Accuracy on known intervention queries "does do(X) change Y?".

Gold answer	Queries	Accuracy
no effect (reverse direction or uncorrelated)	153	1.000
effect (direct or indirect cause)	89	0.933
all	**242**	**0.975**

All six misses are chains $X \to \cdots \to Y$ that pass through an edge whose direction the correlations *cannot identify*. The corresponding indirect claim is valid, because every

equivalent DAG has *some* path, but no single direct edge on that path is certain. The conservative DAG leaves the edge out, so the network sees no path. This is a limit of observational data (T2), not an extraction error. A Phase 2 option is to pass indirect triples to the graph stage as path constraints.

8.3. Why the Bayesian network matters: seeing vs doing

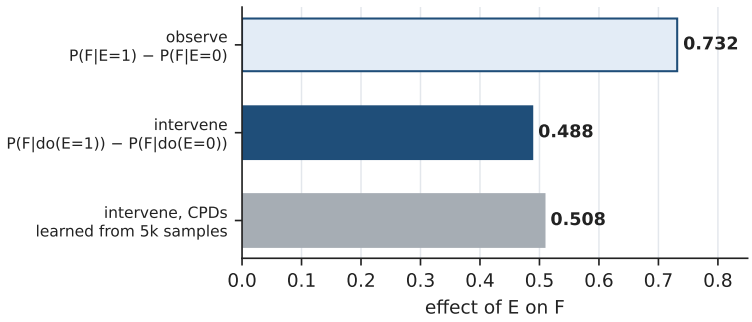


Figure 9: Test premise 20: E→F, with A and D causing both. Observing E overstates its effect on F by 50%. Re-learning the tables from 5,000 simulated samples by maximum likelihood recovers the interventional effect (0.508 vs 0.488).

Figure 9 makes theorem T4 concrete. Because A and D cause both E and F, seeing $E{=}1$ also signals that A or D is probably active. The observational difference (0.732) is therefore much larger than the true causal effect of setting E (0.488). A text extractor can say *that* E causes F; only the network can say *how much*, and keep that separate from confounded association. The final bar shows that the network's probabilities are learnable: with real measurements (e.g. a clinical table), `pgmpy` would fit them instead of using noisy-OR assumptions.

9. Error Analysis and Limitations

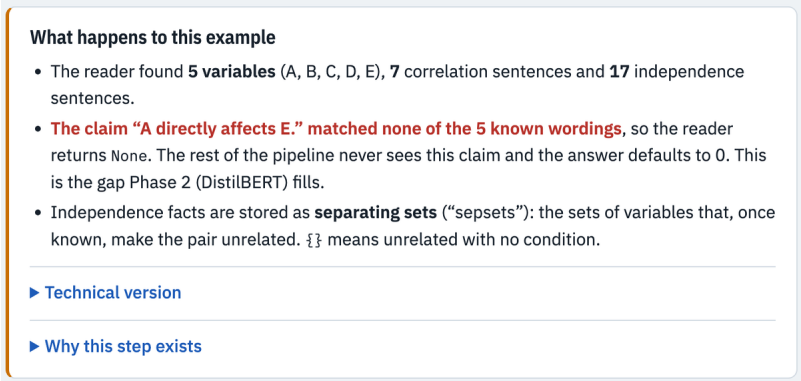


Figure 10: Screenshot from our pipeline tracer: a *true* paraphrased claim ("A directly affects E.") matches none of the five known templates, so the language layer returns nothing and the claim defaults to "not valid".

- **Paraphrase is the open gap.** The regex reader parses 100% of dev, test and refactorization hypotheses but **0%** of paraphrased ones (Fig. 10). The reasoning layer

is never reached, so F1 = 0. TF-IDF does not fail outright, but at 0.25 it is barely useful. This is exactly the "language understanding" half of the brief, and it is what Phase 2 addresses.

- **Shortcut learning in text models.** TF-IDF's collapse under renaming shows that a model that only sees words learns dataset artefacts [11, 12]. We expect fine-tuned transformers to be susceptible too, as Jin et al. report [3], so Phase 2 will test renaming augmentation.
- **Synthetic, closed-world text.** Corr2Cause lists *every* independence. Real abstracts list a few, often with hedging. The PC machinery would then need statistical tests or uncertainty-aware facts. PubMed remains an optional extension.
- **Illustrative probabilities.** Noisy-OR parameters are assumptions, not estimates. The ACE values show the mechanism, not real-world effect sizes (Fig. 9 shows the fitting path).
- **Small test set.** 21 premises (1,162 hypotheses) means one error moves F1 by about 0.003. We therefore also report dev and both robustness sets rather than relying on a single number.

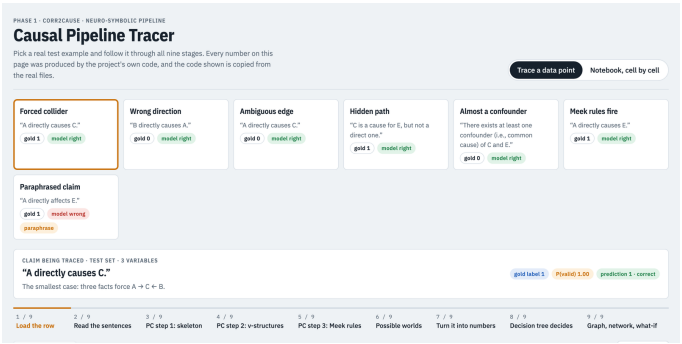
10. Reproducibility and Code Quality

The code is a small Python package with one module per pipeline stage, mirroring Fig. 1:

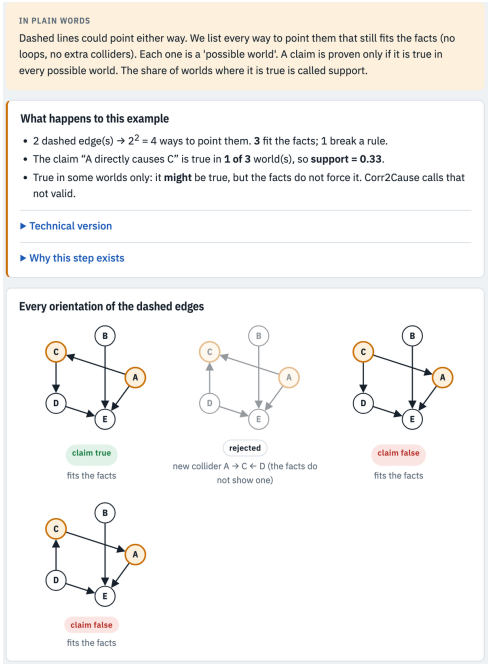
`src/causal_nlp/parsing.py`	**Stage 1**: text → facts / hypothesis triple
`src/causal_nlp/causal_rules.py`	**Stage 2**: PC skeleton, v-structures, Meek R1–R3, equivalent DAGs, support
`src/causal_nlp/features.py`	**Stage 3**: 23-feature table (cached per premise)
`src/causal_nlp/graph_stage.py`	**Stage 5**: acyclic DAG, noisy-OR BN, do-queries, edge metrics
`src/causal_nlp/data.py`	loaders, corrected labels, robustness sets (auto-download)
`notebooks/01_phase1_ml_pipeline.ipynb`	executed Phase 1 deliverable (about 1–2 min on CPU)
`tests/test_pipeline.py`	14 unit tests (parsing, PC, support, semantics, DAG, BN)
`docs/pipeline_visualiser.html`	interactive step-by-step pipeline tracer
`results/`	metrics and test triples saved for the Phase 2 comparison

- **Deterministic.** Pinned dependencies, seed 42, and the official split. Re-running the notebook code for this report reproduced every number above exactly.
- **Tested.** All 14 unit tests pass (`pytest -q`). They cover, for example, a v-structure being found, a chain giving three equivalent DAGs, a cycle and a self-loop being rejected, and confounding making association larger than causation.
- **Swappable layer.** `build_feature_table(inputs, hypothesis_parser=...)` accepts any parser, and `graph_stage` consumes a plain triples table. Phase 2 plugs in without editing downstream code.

- **Explainable.** The pipeline tracer (Fig. 11) follows real test examples through all nine stages, next to the exact source lines that process them, which also serves as viva preparation for every team member.



(a) Choosing a real example and the nine-stage timeline.



(b) Stage 6: the "possible worlds" behind support = 1/3.

Figure 11: Screenshots of the interactive tracer `docs/pipeline_visualiser.html`, which `tools/build_visualiser.py` generates from the project's own code and outputs.

11. Road to Phase 2

Phase 2 keeps the reasoning and graph layers and the evaluation code fixed, and adds two transformer systems, each paired with its Phase 1 counterpart (Table 8).

Table 8: Planned comparison grid. Same splits, same metrics, same graph stage.

	Text only (no reasoning layer)	**Language layer + reasoning layer**
ML (Phase 1)	TF-IDF + LR	Rules + PC + Decision Tree
DL (Phase 2)	**2A**: DistilBERT end-to-end classifier, ± variable-renaming augmentation	**2B**: DistilBERT hypothesis parser → PC + Decision Tree (hybrid)

- **2A** reads `[CLS] hypothesis [SEP] premise [SEP]` (max 512 tokens, truncating the premise tail) and is expected to beat TF-IDF in distribution, but to inherit the renaming weakness unless names are augmented.

- **2B** is the hybrid the brief rewards: DistilBERT only maps a hypothesis to `(relation, x, y)`. It is trained on template hypotheses plus our own hand-written paraphrases, with the official paraphrase wordings held out. Paraphrase F1 should rise from 0 towards the parser's accuracy while in-distribution and renaming scores stay at Phase 1 levels.
- Final table: dev / test / refactorization / paraphrase F1, edge precision and recall, query accuracy, training time and parameter count, plus an error analysis of 20 failures per DL system.

12. Conclusion

Phase 1 shows that separating language from reasoning is effective. A small set of regex rules, a faithful implementation of the PC algorithm, and a depth-4 decision tree solve Corr2Cause almost perfectly (test F1 0.997). They are unaffected by variable renaming, recover every gold cause edge, and answer 97.5% of intervention queries. The Bayesian network adds what extraction cannot: guaranteed acyclicity, quantified effects, and a principled distinction between observing and intervening. The remaining weakness is purely linguistic. When the wording changes, the rules cannot read the claim, so Phase 2 brings in DistilBERT for exactly that job while the reasoning layer stays as it is.

13. Team Contributions

Member	Primary responsibility
Krishnadevan Manju Liby	Project lead: pipeline design, graph stage, Bayesian network and do-queries; pipeline tracer; report (Sections 8 and 10)
Hemanth Tenneti	Data loading, label correction, leakage checks and EDA (Section 4)
Arina Ali	Feature engineering, classifiers and ablation (Section 7)
Manan Bansal	Reasoning layer: PC algorithm, Meek rules, equivalent DAGs and support; graph-stage tests (Section 5)
Aaloke Eppalapalli	Language layer: entity schema and regex extractor; robustness sets (Section 6)

References

[1] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[3] Z. Jin, J. Liu, Z. Lyu, S. Poff, M. Sachan, R. Mihalcea, M. Diab, and B. Schölkopf, "Can large language models infer causation from correlation?" in *Proc. ICLR*, 2024.

[4] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.

[5] C. Meek, "Causal inference and causal explanation with background knowledge," in *Proc. UAI*, 1995, pp. 403–410.

[6] T. Verma and J. Pearl, "Equivalence and synthesis of causal models," in *Proc. UAI*, 1990, pp. 255–270.

[7] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[9] C. S. G. Khoo, S. Chan, and Y. Niu, "Extracting causal knowledge from a medical database using graphical patterns," in *Proc. ACL*, 2000, pp. 336–343.

[10] R. Girju, "Automatic detection of causal relations for question answering," in *Proc. ACL Workshop on Multilingual Summarization and Question Answering*, 2003, pp. 76–83.

[11] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. Bowman, and N. A. Smith, "Annotation artifacts in natural language inference data," in *Proc. NAACL-HLT*, 2018, pp. 107–112.

[12] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.

[13] E. Kıcıman, R. Ness, A. Sharma, and C. Tan, "Causal reasoning and large language models: Opening a new frontier for causality," arXiv:2305.00050, 2023.

[14] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: The 3rd wave," *Artificial Intelligence Review*, vol. 56, pp. 12387–12406, 2023.

[15] A. Ankan and A. Panda, "pgmpy: Probabilistic graphical models using Python," in *Proc. SciPy*, 2015, pp. 6–11.

[16] Corr2Cause corrected labels, Hugging Face dataset `zluke/corr2cause-corrected`, https://huggingface.co/datasets/zluke/corr2cause-corrected.